

Linked Lists in Python

From Memory Fundamentals to Advanced Implementation

Linked List Structure



Table of Contents

1. Memory Fundamentals & Arrays
2. Why Linked Lists?
3. Node Structure & Singly Linked Lists
4. Operations: Traversal, Insert, Delete
5. Doubly Linked Lists
6. Array vs Linked List Comparison
7. Reversal Logic & Full Python Implementation

1. Memory Fundamentals

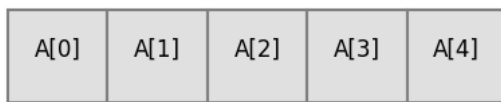
Linear Address Space: Computer memory is like a long street of numbered mailboxes. Each box (byte) has a unique address.

Python References: In Python, variables don't "hold" values; they hold *pointers* (addresses) to objects in memory.

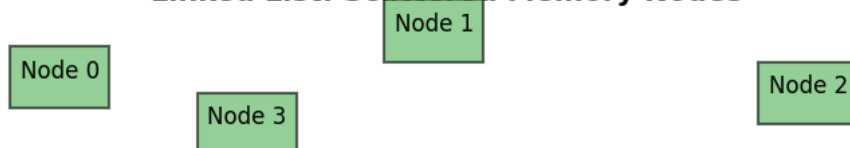
2. Arrays & Memory Layout

Arrays (or Python Lists) use **Contiguous Allocation**. All elements are stored side-by-side in one solid block of memory.

Array: Contiguous Memory



Linked List: Scattered Memory Nodes



The Insertion Problem

To insert an element at the beginning of an array, every other element must be **shifted** one position to the right. **Time Complexity: $O(n)$**

3. Why Linked Lists Were Invented

Linked Lists solve two main problems of arrays:

1. **Memory Fragmentation:** If memory is scattered, you might not find one big block for an array, even if you have enough total space. Linked lists use tiny scattered slots.
2. **Dynamic Scaling:** Arrays have fixed sizes. Linked lists grow and shrink perfectly as needed without costly "reallocations."

4. Node Structure

The "Atom" of a linked list is the **Node**. It consists of two parts:

Data
(The Value)

Next
(Pointer/Reference)

```
class Node:
    def __init__(self, data):
        self.data = data      # Green Box
        self.next = None     # Yellow Box
```

5. Singly Linked Lists

A chain of nodes where each node points to the next. The list starts at a **Head** and ends when a node points to **None**.

6. Traversal (The Pointer Movement)

To visit every node, we use a "temporary" pointer called `current`.

```
current = head
while current is not None:
    print(current.data)
    current = current.next # This moves the pointer!
```

Visualizing Traversal:

1. `curr` starts at Head.
2. `curr = curr.next` jumps to the second node.
3. Repeat until `curr` is None.

7. Insert & Delete Operations

Insert at Head $O(1)$

1. Create new node.
2. Point new node's **Next** to current **Head**.
3. Update **Head** to be the new node.

Delete by Value $O(n)$

To delete Node B: Simply make Node A's **Next** point directly to Node C. Node B is "bypassed" and Python's Garbage Collector handles the rest.

[A] -> [B] -> [C] ---(Delete B)---> [A] -----> [C]

8. Doubly Linked Lists

Each node now has **three** fields: **Prev**, **Data**, and **Next**. This allows of moving backward in the list.

Prev

Data

Next

```
class DNode:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None
```

Pros: Can traverse backwards, deletion is easier if node is known.

Cons: Extra memory for prev pointer.

9. Array vs Linked List

Operation	Array (Python List)	Linked List
Indexing (Access)	O(1)	O(n)
Insert at Start	O(n)	O(1)
Insert at End	O(1)* (Amortized)	O(1) (if Tail exists)
Memory Locality	Excellent (Cache Friendly)	Poor

10. Reversing a Linked List

This is the classic interview question. It requires **three pointers**: prev, curr, and next_node.

```
prev = None
curr = head
while curr:
    next_node = curr.next # 1. Save next
    curr.next = prev      # 2. Reverse Link!!
    prev = curr           # 3. Move prev forward
    curr = next_node      # 4. Move curr forward
head = prev
```

11. Full Python Implementation (Part 1)

```
class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node

    def prepend(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
```

12. Full Python Implementation (Part 2)

```
def delete_node(self, key):
    curr = self.head
    if curr and curr.data == key:
        self.head = curr.next
        curr = None
        return
    prev = None
    while curr and curr.data != key:
        prev = curr
        curr = curr.next
    if curr is None: return
    prev.next = curr.next
    curr = None

def display(self):
    elems = []
    curr = self.head
    while curr:
        elems.append(str(curr.data))
        curr = curr.next
    print(" -> ".join(elems) + " -> None")
```

Definition

A linear data structure where elements are not stored contiguously, but linked via pointers.

When to use?

When you need frequent insertions/deletions at the start or mid-list and don't know the exact data size.

Common Pitfalls

- Losing the **Head** reference (you lose the whole list!).
- Infinite loops (forgetting to move the pointer: `curr = curr.next`).
- Dereferencing None (trying to access `.next` on a null pointer).